Документация к GPU Sandbox

Содержание

1 Назначение программы **2**

2 Архитектура проекта **2**

2.1 Память 2

2.2 Кэш-память 3

2.3 Ядро процессора 3

2.3.1 Регистры 4

2.3.2 Программный счётчик 4

2.3.3 Флаги состояния 4

2.3.4 Состояния ядра 4

3 Инструкции виртуального процессора **4**

3.1 Параметры инструкций 5

3.2 Поддерживаемые операции 6

4 Симулятор выполнения **6**

5 Проверка корректности входных данных **8**

6 Статистика выполнения **8**

7 Отладочный вывод **9**

8 Сброс и инициализация **9**

9 Тестирование **10**

1 Назначение программы

GPU Sandbox представляет собой программную модель вычислительного ядра, памяти и набора инструкций, реализованную на языке C. Цель проекта — изучение основных принципов работы процессора: адресация памяти, выполнение операций в регистрах, обработка состояний ядра, проверка корректности входных данных и симуляция исполнения программы.

На текущем этапе проект уже содержит не только базовые вычисления, но и элементы условных переходов, проверки ошибок, таймаутов, отладки и автоматического тестирования[1].

2 Архитектура проекта

Модель содержит вычислительные ядра, общую память, программу из набора инструкций и двухуровневый кэш данных.

Таблица 1: Ключевые компоненты архитектуры

Компонент	**Назначение**
`Memory`	Общая виртуальная память, хранящая 32-битные значения и обеспечивающая чтение/запись по адресам.
`Cache`	Двухуровневая буферизация доступа к памяти: отдельный кэш L1 для каждого ядра и общий кэш L2.
`Core`	Виртуальное вычислительное ядро с регистрами, счётчиком команд, флагом сравнения и состоянием выполнения.
`Instruction`	Структура команды, задающая тип операции и аргументы `a`, `b`, `c`.
`Simulator`	Планировщик задач на нескольких виртуальных ядрах с общей памятью и общим лимитом тактов.
`Statistics`	Набор счётчиков выполненных инструкций, операций и ошибок для каждой задачи.

2.1 Память

`Memory` — это структура виртуальной памяти, хранящая данные в массиве `data[MEMORY_SIZE]`.

Память используется для хранения значений типа `uint32_t`, которые считываются и записываются в ходе выполнения программы. На данный момент размер памяти определяется константой `MEMORY_SIZE = 1024 * 1024`.

Операции чтения и записи реализованы следующим образом:

- `memory_write(Memory *memory, uint32_t address, uint32_t value)` — запись 32-битного значения в ячейку памяти;

[1] Подробнее об устройстве ядра, памяти и логике выполнения см. в разделе *Архитектура проекта*.

- `memory_read(const Memory *memory, uint32_t address)` — чтение 32-битного значения из памяти;
- `memory_reset(Memory *memory)` — очистка памяти до нуля.

Проверка допустимости адреса выполняется через специальный валидатор, который запрещает обращение за пределы памяти.

2.2 Кэш-память

Кэш хранит копии блоков общей памяти и обслуживает операции `OP_LOAD` и `OP_STORE`. В симуляторе у каждого ядра есть собственный кэш первого уровня L1 на 32 набора, а все ядра используют общий кэш второго уровня L2 на 128 наборов. Каждый набор содержит два пути, а строка кэша хранит четыре 32-битных слова. Таким образом, ёмкость L1 одного ядра составляет 256 слов, а L2 — 1024 слова.

Адрес слова раскладывается на номер блока, индекс набора, тег и смещение слова внутри строки:

- номер блока равен `address / CACHE_LINE_WORDS`;
- индекс набора равен остатку от деления номера блока на число наборов;
- тег равен целой части от деления номера блока на число наборов;
- смещение внутри строки равно `address % CACHE_LINE_WORDS`.

При промахе свободный путь используется первым. Если оба пути заняты, выбирается путь, отмеченный в наборе как наименее недавно использованный. После обращения или заполнения набора этот признак обновляется, поэтому замещение реализует двухпутевую LRU-политику. При загрузке строки читаются все четыре слова соответствующего блока.

Чтение выполняется сначала в L1. При попадании слово возвращается непосредственно из L1; при промахе проверяется L2. Если блок найден в L2, вся строка копируется в L1. Если промах произошёл и в L2, блок загружается из общей памяти в L2, а затем копируется в L1.

Запись реализована по сквозной политике: изменённое слово сразу записывается в L1 и общую память. Затем обновляется соответствующий блок в L2; при его отсутствии блок загружается туда из уже обновлённой памяти. Поэтому общая память остаётся актуальной и общей для всех ядер.

В структуре `CacheStatistics` отдельно для каждого экземпляра кэша учитываются попадания `hits`, промахи `misses` и вытеснения `evictions`. Промах учитывается при проверке набора, а вытеснение — когда при заполнении заменяется уже действительная строка. Эти счётчики не входят в статистику отдельной задачи `Statistics`.

2.3 Ядро процессора

`Core` — структура вычислительного ядра. В ней хранятся регистры, программный счётчик, текущее состояние и флаг сравнения.

2.3.1 Регистры

`registers[REGISTERS_COUNT]` — массив регистров, который используется для промежуточных вычислений и передачи данных между инструкциями.

Количество регистров задаётся константой `REGISTERS_COUNT`, который, на данный момент, равен 8.

2.3.2 Программный счётчик

`pc` (программный счётчик) — счётчик, который хранит индекс текущей выполняемой команды. После выполнения инструкции счётчик изменяется в соответствии с состоянием ядра.

2.3.3 Флаги состояния

Внутри ядра присутствует переменная `flag`, которая используется для результатов сравнения. В проекте применяются следующие значения:

- `SAME` — значения равны;
- `GREATER` — первое значение больше второго;
- `LESS` — первое значение меньше второго.

2.3.4 Состояния ядра

`current_state` управляет поведением выполнения программы. Набор состояний представлен в перечислении `State`:

1. `NORMAL` — обычный режим работы ядра;
2. `JUMP` — выполнение перехода на другую инструкцию;
3. `HALT` — завершение работы программы;
4. `ERROR` — аварийное состояние при ошибке выполнения;
5. `TIMEOUT` — превышен лимит количества циклов.

При возникновении ошибки, например, вызове некорректного регистра, адреса памяти или неизвестного оператора, состояние переводится в `ERROR`. Если же программа выполняется слишком долго, то срабатывает таймаут по переменной `MAX_CYCLES`.

3 Инструкции виртуального процессора

Структура инструкции представлена типом `Instruction`[2]. Максимальный размер программы задан константой `MAX_PROGRAM_SIZE = 256`; в текущей реализации эта константа объявлена, но отдельно не проверяется при запуске.

[2]Подробное описание набора операций и их логики работы представлено в таблице инструкций.

```
typedef struct{
    Opcode opcode;
    uint32_t a;
    uint32_t b;
    uint32_t c;
} Instruction;
```

В проекте используются следующие значения опкодов:

1. OP_LOAD
2. OP_STORE
3. OP_ADD
4. OP_SUB
5. OP_MUL
6. OP_DIV
7. OP_HALT
8. OP_JUMP
9. OP_CMP
10. OP_JIF

3.1 Параметры инструкций

В проекте применяется следующий порядок использования операндов:

- a — целевой регистр или регистр с адресом памяти, в зависимости от инструкции;
- b — регистр с адресом памяти, исходный регистр или второй аргумент, в зависимости от инструкции;
- c — второй источник данных при арифметических операциях.

3.2 Поддерживаемые операции

Таблица 2: Набор инструкций

Операция	**a**	**b**	**c**	**Назначение**
`OP_LOAD`	регистр назначения	регистр с адресом	—	Чтение значения из `memory[reg[b]]` в `reg[a]`
`OP_STORE`	регистр с адресом	регистр со значением	—	Запись значения `reg[b]` в ячейку `memory[reg[a]]`
`OP_ADD`	регистр назначения	регистр 1	регистр 2	Сложение `reg[b] + reg[c]` и сохранение в `reg[a]`
`OP_SUB`	регистр назначения	регистр 1	регистр 2	Вычитание `reg[c]` из `reg[b]` и сохранение в `reg[a]`
`OP_MUL`	регистр назначения	регистр 1	регистр 2	Умножение `reg[b] * reg[c]` и сохранение в `reg[a]`
`OP_DIV`	регистр назначения	регистр 1	регистр 2	Деление `reg[b] / reg[c]` с проверкой на деление на ноль
`OP_JUMP`	индекс инструкции	—	—	Безусловный переход к инструкции с индексом `a`
`OP_CMP`	регистр 1	регистр 2	—	Сравнение значений регистров и установка флага `flag`
`OP_JIF`	индекс инструкции	ожидаемый флаг	—	Условный переход: если `b == core->flag`, ядро переходит к инструкции с индексом `a`
`OP_HALT`	—	—	—	Завершение выполнения программы и перевод ядра в состояние `HALT`

Важно: фактическая реализация проекта допускает и проверяет корректность входных индексов и адресов, поэтому многие операции в случае неверного операнда переводят систему в состояние `ERROR`.

4 Симулятор выполнения

Модуль `simulator` выполняет задачи на нескольких виртуальных ядрах. Программа представлена структурой `Program` (указатель на инструкции и их количество),

а структура `Task` хранит программу, начальные значения регистров, статус и статистику задачи. Структура `CoreContext` связывает виртуальное ядро с назначенной задачей и её кэшем L1. Структура `Simulator` содержит массивы ядер и задач, общую память, общий кэш L2, индекс следующей задачи и глобальное число тактов.

Планировщик назначает задачи свободным ядрам в порядке их расположения в массиве задач. За один глобальный такт каждое занятое ядро исполняет не более одной инструкции. Операции выполняются последовательно в порядке обхода массива ядер. Память и кэш L2 общие для всех задач, кэш L1 закреплён за ядром. Начальные регистры задачи копируются в ядро после его сброса.

Для отдельной задачи возможны статусы `FREE`, `WORKING`, `DONE`, `TASK_ERROR` и `TASK_TIMEOUT`. Выполнение задачи заканчивается при достижении конца программы, инструкции `HALT` или ошибке. Общая симуляция завершается, когда обработаны все задачи либо достигнут лимит `MAX_CYCLES`. При общем таймауте ещё выполняющиеся задачи получают статус `TASK_TIMEOUT`.

Модуль запускает симуляцию до тех пор, пока не будет выполнен один из следующих сценариев:[3]

- достигнут конец программы;
- ядро перешло в состояние `HALT`;
- возникла ошибка;
- число выполненных циклов превысило лимит `MAX_CYCLES`.

Поле `Simulator.cycles` хранит количество глобальных тактов всей симуляции. Константа `MAX_CYCLES`, на данный момент, равна 1 000 000. Дополнительно, каждая задача ведёт собственный счётчик выполненных инструкций и тактов в структуре статистики.

Схема выполнения выглядит так:

1. планировщик назначает свободным ядрам очередные задачи;
2. каждое занятое ядро выполняет одну инструкцию через `step_core` и `execute`;
3. инструкции чтения и записи обращаются к L1, затем при необходимости к общему L2 и основной памяти;
4. после инструкции обновляются счётчики статистики, программный счётчик и статус задачи;
5. завершившиеся или ошибочные задачи освобождают ядра;
6. после обхода ядер увеличивается общий счётчик тактов `Simulator.cycles`;
7. при достижении общего лимита оставшиеся активные задачи получают статус `TASK_TIMEOUT`.

[3]Более подробно тесты многозадачной симуляции описаны в разделе *Тестирование*.

5 Проверка корректности входных данных

Для контроля ошибок проект использует набор валидаторов, реализованных в модуле инструкций[4]. Основные проверки:

- `checker_destination` — проверка допустимого индекса целевого регистра;
- `checker_address_register` — проверка адресного регистра;
- `checker_source_register` — проверка регистра-источника;
- `checker_condition_register` — проверка регистра условия;
- `checker_target_register` — проверка регистра перехода;
- `checker_memory_address` — проверка корректности адреса памяти;
- `checker_instruction_index` — проверка индекса инструкции в пределах программы.

В случае некорректных данных ядро переводится в состояние `ERROR`, а в консоль выводится сообщение о причине сбоя.

6 Статистика выполнения

Для каждой задачи симулятор хранит структуру `Statistics`. Счётчики операций увеличиваются при успешном выполнении соответствующей инструкции; счётчик ошибок увеличивается при обнаружении ошибки валидации, делении на ноль или неизвестном опкоде.

[4] Подробнее о логике ошибок и проверках см. раздел *Симулятор выполнения*.

Таблица 3: Счётчики структуры `Statistics`

Поле	**Тип**	**Назначение**
`cycles`	`uint32_t`	Число тактов, затраченных на выполнение задачи.
`instruction_executed`	`uint32_t`	Общее число исполненных инструкций задачи.
`loads`, `stores`	`uint32_t`	Число успешно выполненных операций чтения и записи памяти.
`adds`, `subs`	`uint32_t`	Число успешно выполненных сложений и вычитаний.
`muls`, `divs`	`uint32_t`	Число успешно выполненных умножений и делений.
`jumps`	`uint32_t`	Число выполненных безусловных переходов.
`conditional_jumps`	`uint32_t`	Число условных переходов, которые действительно изменили адрес выполнения.
`checks`	`uint32_t`	Число успешно выполненных сравнений инструкцией `OP_CMP`.
`errors`	`uint32_t`	Число зарегистрированных ошибок при выполнении инструкций и проверке операндов.

7 Отладочный вывод

Проект содержит набор функций для вывода состояния виртуального процессора и симуляции:[5]

- `print_registers` — печать всех регистров;
- `print_state` — печать текущего состояния ядра;
- `print_program` — отображение программы с операциями и аргументами;
- `print_statistics` — вывод статистики отдельной задачи;
- `print_simulator` — вывод общего числа тактов, состояний ядер и статусов задач с их статистикой.

8 Сброс и инициализация

Для повторного использования модели предусмотрены функции сброса:

- `core_reset(Core *core)` сбрасывает регистры в ноль, обнуляет счётчик `pc` и возвращает состояние ядра в `NORMAL`;

[5] Функции отладки напрямую связаны с режимами работы ядра, описанными в разделе *Архитектура проекта*.

- `memory_reset(Memory *memory)` очищает память;
- `cache_reset(Cache *cache)` очищает строки и счётчики кэша, сохраняя число наборов. При запуске симуляции кэши L1 и L2 заново инициализируются функцией `cache_init`.

Это позволяет запускать несколько тестовых сценариев последовательно на одном и том же экземпляре модели.

9 Тестирование

В папке `tests` находятся отдельные программы, проверяющие корректность работы ключевых компонентов и обновлённой многозадачной симуляции:[6]

- `test_core.c` — проверяет сброс ядра;
- `test_memory.c` — проверяет запись, чтение и очистку памяти;
- `test_cache.c` — проверяет декодирование адресов, попадания и промахи, выбор пути, вытеснение, заполнение строк, запись, инвалидацию и чтение через L2;
- `test_instruction.c` — проверяет основные инструкции, флаги сравнения и обработку ошибок;
- `test_debug.c` — проверяет функции вывода состояния и симулятора;
- `test_simulator.c` — проверяет распределение нескольких задач между ядрами, результаты в общей памяти, обработку ошибочной задачи и таймаут.

Автотесты написаны через макросы `assert`, что позволяет проверять корректность поведения программы на уровне условий и побочных эффектов.

[6] См. также раздел *Проверка корректности входных данных*.